The Dining Philosophers

vs.

The Cigarette Smokers

Kiatisak Markmeeshap	67070501005
Tithinan Sobking	67070501018
Worawut Sereethai	67070501040
Chanya Poolketkij	67070501058

1. The Cigarette Smokers problem (Patil, 1971)

- Making a cigarette needs **tobacco**, **paper** and **matches**.
- Three smokers each hold an unlimited supply of *one* ingredient.
- An **agent** (a dealer that supplies ingredients, like the OS) holds all three. It repeatedly puts two different ingredients on the table, then waits for a smoker to finish.
- The smoker holding the third ingredient should take both, smoke, and wake the agent.
- **Constraint:** the agent's code is fixed; only the smokers may be written.

Patil posed it to argue that semaphores (P/V) are too weak to solve it without conditional statements [1].

Agent, as given (one of three; the others supply the other two pairs) [3]

```
agentSem = Semaphore(1)
tobacco = Semaphore(0)
paper   = Semaphore(0)
match   = Semaphore(0)

// Agent A
agentSem.wait()
tobacco.signal()
paper.signal()
```

Agents B and C signal `paper+match` and `tobacco+match`.

2. Why the obvious solution deadlocks

```
// smoker with matches
tobacco.wait()
paper.wait()
agentSem.signal()

// smoker with paper
tobacco.wait()
match.wait()
agentSem.signal()

// smoker with tobacco
paper.wait()
match.wait()
agentSem.signal()
```

Interleaving. The agent puts out *tobacco* + *paper* (meant for the matches smoker).

1. The *paper* smoker takes the tobacco, then blocks on `match`.
2. The *tobacco* smoker takes the paper, then blocks on `match`.
3. The *matches* smoker blocks on `tobacco`; the agent blocks on `agentSem`.

No one can proceed: a **deadlock** [3].

Each semaphore says only "this ingredient is here". A smoker cannot tell whether an ingredient is meant for it until it has seen *both*, so it grabs one and holds it while waiting (*hold and wait*).

3. A solution: pushers decide who smokes

```
mutex = Semaphore(1)
isTobacco = isPaper = isMatch = False
tobaccoSem = paperSem = matchSem = Semaphore(0)

// Pusher A: one per ingredient
tobacco.wait()
mutex.wait()
if isPaper:
    isPaper = False
    matchSem.signal()
elif isMatch:
    isMatch = False
    paperSem.signal()
else:
    isTobacco = True
mutex.signal()

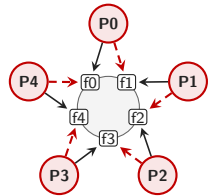
// smoker with tobacco
tobaccoSem.wait()
makeCigarette(); agentSem.signal(); smoke()
```

- A helper thread (*pusher*) waits on each ingredient semaphore and records it in shared state under a mutex.
- The pusher that sees the *second* ingredient knows the pair, and signals exactly the one smoker that can use it.
- Smokers wait on a single semaphore, so none ever holds a partial pair. The agent is unchanged.

Downey's form [3]. Parnas showed Patil's claim holds only under strong restrictions on P and V, and solved the problem without conditionals [2].

4. The Dining Philosophers (Dijkstra, 1971)

- Five philosophers alternate between thinking and eating; one fork lies between each pair.
- Eating needs both the left and the right fork.
- If all five take their left fork at once, each waits forever for the right one: a **circular wait** [4], [6].
- Standard fixes [3], [5], [6]:
  - order the forks (one philosopher takes the right first);
  - seat at most four at a time;
  - take both forks only when both are free (Dijkstra's state-based solution).



All take the left fork:
— holds
--- waits for
a cycle: deadlock

Breaking the circular wait (Lecture 10) [6]

```
int left(int p)  { return p; }
int right(int p) { return (p + 1) % 5; }

void getforks(int p) {
    if (p == 4) {
        sem_wait(&forks[right(p)]);
        sem_wait(&forks[left(p)]);
    } else {
        sem_wait(&forks[left(p)]);
        sem_wait(&forks[right(p)]);
    }
}
```

5. The two problems side by side

	Dining Philosophers	Cigarette Smokers
Resources	Five forks, *reusable*: returned after eating	Three ingredients, *consumable*: used up, then re-supplied by the agent
Who needs what	Fixed: philosopher p always needs forks p and $p+1$	Decided at run time by the agent's random choice of pair
Symmetry	All processes identical	Three different smokers, plus an agent we may not change
What we control	All of the code	Only the smokers (and any helpers we add)
Deadlock cause	Circular wait on forks held by neighbours	Taking an ingredient meant for someone else, then waiting for one that never comes
Typical fix	Order resources, limit diners, or take both forks at once	Collect the pair first, then wake the right smoker (pushers)
What it teaches	Deadlock and starvation in resource allocation	How expressive semaphores are; synchronising on combined state

6. Why the philosophers' fixes do not save the smokers

- **Resource ordering.** Order the ingredients tobacco $<$ paper $<$ match. The code on slide 2 already acquires in that order, and it still deadlocks. Ordering breaks a cycle of processes *holding* resources; here the missing ingredient is held by no one: the agent never produces it.
- **Acquire both atomically.** Wrapping both waits in one mutex (Lecture 10's hold-and-wait prevention) is worse: the first smoker in waits for an ingredient that is not on the table, while holding the mutex every other smoker needs.
- **What does carry over.** Dijkstra's own solution and the pushers share one shape: record state under a mutex, decide who can proceed, and signal that process's private semaphore [3], [4]. The decision is made where the whole state is visible, not by each process grabbing one resource at a time.

7. Conclusion

- Both problems deadlock when a process takes one resource and waits for a second.
- In the **Dining Philosophers** the second resource *exists* but is held by a neighbour, so breaking the circular wait is enough.
- In the **Cigarette Smokers** the second resource may *never be produced* for that smoker, so no order of waits helps; the pair must be identified before anyone commits to it.
- The **Cigarette Smokers** also fixes part of the system (the agent), which is the realistic case: an OS allocates resources without knowing which process will use them.
- Both are solved by the same pattern: shared state under a mutex, plus one semaphore per process to wake exactly the one that can run.

References

[1] S. S. Patil, "Limitations and capabilities of Dijkstra's semaphore primitives for coordination among processes," Computation Structures Group Memo 57, Project MAC, MIT, Cambridge, MA, Feb. 1971.

[2] D. L. Parnas, "On a solution to the cigarette smoker's problem (without conditional statements)," *Commun. ACM*, vol. 18, no. 3, pp. 181–183, Mar. 1975, doi:10.1145/360680.360709.

[3] A. B. Downey, *The Little Book of Semaphores*, 2nd ed., v2.2.1. Green Tea Press, 2016, Sec. 4.4 (Dining philosophers) and Sec. 4.5 (Cigarette smokers problem).

[4] E. W. Dijkstra, "Hierarchical ordering of sequential processes," *Acta Informatica*, vol. 1, no. 2, pp. 115–138, 1971, doi:10.1007/BF00289519.

[5] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, v1.10. Arpaci-Dusseau Books, 2023, Ch. 31 (Semaphores).

[6] CPE 333 Operating Systems, Lecture 10: Resource Allocation and Deadlock, 1/2026, slides 9–21.